

# Síntese de Programas com CEGIS e Restrições Anti-Overfitting no ARC-AGI: Uma Análise Estrutural e Estatística

Fabio Luiz da Costa Cieslak<sup>\*1</sup>, Izadora Candotti de Oliveira<sup>\*1</sup>, Matheus Machado<sup>\*1</sup>

Universidade Federal do Rio Grande do Sul<sup>1</sup>

## Abstract

Resolver problemas visuais e lógicos do benchmark ARC-AGI por meio da geração automática de código em Python usando modelos de linguagem (LLMs) é um desafio central para a inteligência artificial. Um problema recorrente nessa abordagem é o ajuste excessivo aos exemplos de treino (*overfitting*): o modelo cria soluções que funcionam apenas para os exemplos mostrados, mas falham no teste. Neste trabalho, investigamos o método de Síntese Indutiva Guiada por Contraexemplos (CEGIS) e avaliamos uma variação com regras anti-ajuste (*AntiCheat*), que proíbe regras pontuais para pixels específicos e exige que o modelo explique em texto a lógica geral antes de escrever o código. Avaliamos 8 modelos de linguagem em 100 tarefas desafiadoras do ARC-AGI. Os resultados mostram que a inclusão das regras anti-ajuste não alterou a acurácia global de forma estatisticamente significativa (acurácia de 50,4% no CEGIS contra 49,8% no AntiCheat, com  $p = 0,5515$ ). A análise qualitativa do código gerado também indicou que ambos os métodos produzem programas estruturalmente semelhantes e falham pelos mesmos motivos, principalmente pela incapacidade de encontrar a lógica correta dentro do limite de tentativas. Por outro lado, a análise estrutural revelou que as tarefas seguem uma escala de dificuldade unidimensional quase perfeita de Guttman ( $CR = 0,9738$ ) e que os códigos com sobreajuste violam a Navalha de Occam, sendo 1,61 vezes maiores do que os programas que acertam de primeira ( $p = 5,15 \times 10^{-9}$ ). Por fim, observamos que 86,0% das correções bem-sucedidas do CEGIS ocorrem até a terceira iteração.

## 1 Introdução

O *Abstraction and Reasoning Corpus* (ARC-AGI) (Chollet 2019) foi criado para medir a capacidade de sistemas artificiais de aprender novas regras a partir de poucos exemplos. Cada tarefa do ARC apresenta grades bidimensionais com números que representam cores e desafia o sistema a descobrir a regra de transformação para gerar a grade de saída a partir de uma grade de entrada inédita.

Uma das abordagens mais promissoras para resolver o ARC é a síntese de programas com Modelos de Linguagem (LLMs) (Greenblatt et al. 2024). Em vez de gerar a grade

diretamente como texto, o modelo escreve uma função em Python (`transform(grid)`) que executa a lógica da tarefa. Para melhorar o código gerado, é comum utilizar a Síntese Indutiva Guiada por Contraexemplos (CEGIS) (Solar-Lezama 2008): o código é testado nos exemplos de treino; se falhar em algum exemplo (chamado de contraexemplo), o erro é devolvido ao modelo para que ele faça uma correção iterativa.

No entanto, essa abordagem enfrenta um desafio crítico: o ajuste excessivo aos dados de treino (*overfitting*). O modelo pode consertar o código inserindo regras pontuais (*hardcoding*) ou condicionais especiais para os exemplos de treino conhecidos, sem aprender o padrão lógico real. Quando o programa resultante é executado na grade de teste, ele falha.

Para enfrentar esse problema, avaliamos neste trabalho uma estratégia com regras explícitas contra trapaças lógicas, denominada *AntiCheat*. Essa estratégia impõe duas restrições ao modelo no momento do feedback: (1) proíbe o uso de condicionais direcionadas a coordenadas fixas ou índices específicos de exemplos; e (2) exige que o modelo descreva em linguagem natural a regra abstrata geral antes de escrever a nova função em Python.

A hipótese central investigada neste trabalho é: *adicionar regras que forcem explicações prévias e proíbem regras pontuais melhora a generalização e a acurácia dos modelos no ARC-AGI?*

Compreender o impacto dessas restrições é fundamental para a engenharia de sistemas autônomos baseados em código, pois ajuda a distinguir o raciocínio genuíno da memorização superficial em modelos de linguagem.

## Principais Contribuições

As principais contribuições deste artigo são:

- Avaliação estatística rigorosa de CEGIS versus AntiCheat:** Realizamos 800 avaliações pareadas (8 modelos em 100 tarefas) utilizando testes de permutação e o teste binomial exato para comparar diretamente a acurácia das duas abordagens.
- Inspeção qualitativa do código gerado:** Analisamos as diferenças reais no código e os motivos de falha de cada abordagem, verificando se as restrições provocam mudanças significativas na lógica produzida.
- Validação da Escala de Guttman:** Demonstramos que o desempenho dos modelos no ARC-AGI forma uma es-

<sup>\*</sup>These authors contributed equally.

cala psicométrica unidimensional válida ( $CR = 0,9738$ ), indicando que modelos mais fortes absorvem quase integralmente as capacidades dos modelos mais simples.

4. **Comprovação empírica da Navalha de Occam:** Mostramos, a partir da árvore sintática abstrata (AST), que soluções com sobreajuste são 1,61 vezes mais extensas do que soluções corretas ( $p = 5,15 \times 10^{-9}$ ).
5. **Determinação do horizonte ótimo de iterações:** Mapear a recuperação do CEGIS e mostramos que 86% de todas as correções acontecem até a terceira iteração.

## Resultados Obtidos

Adiantando os resultados, constatamos que a estratégia AntiCheat **não produz diferença estatisticamente significativa** na acurácia global em relação ao CEGIS padrão (50,4% contra 49,8%, com  $p = 0,5515$ ). Além disso, a inspeção minuciosa do código gerado revelou que, mesmo em nível qualitativo, não há diferença substancial: em mais de 88% dos casos de discordância, a estratégia perdedora falha simplesmente por não conseguir encontrar um algoritmo válido para os exemplos de treino, e não por tentar criar regras pontuais.

## Estrutura do Artigo

O restante do artigo está organizado da seguinte forma: a Seção 2 aborda os trabalhos relacionados. A Seção 3 descreve a abordagem do CEGIS e o mecanismo AntiCheat. A Seção 4 apresenta a metodologia experimental. A Seção 5 detalha os resultados empíricos e estruturais. Por fim, a Seção 6 discute as conclusões e os trabalhos futuros.

## 2 Trabalhos Relacionados

### 2.1 O Benchmark ARC-AGI e a Síntese de Programas

A *Abstraction and Reasoning Corpus* (ARC) foi proposto por Chollet (Chollet 2019) com o objetivo explícito de medir a inteligência geral artificial em contraste com a simples memorização de bases massivas de dados. No ARC, cada tarefa exige a indução de conceitos como simetria, conectividade, trajetória e contagem a partir de 2 a 5 pares de treino.

Trabalhos recentes mostram que a geração direta de texto pixel a pixel por LLMs atinge desempenho limitado, ao passo que a síntese de programas em linguagens como Python (Greenblatt et al. 2024) permite ao modelo delegar a execução precisa e determinística a um interpretador. Isso transforma o desafio do ARC em um problema clássico de síntese de programas a partir de exemplos.

### 2.2 Síntese Indutiva Guiada por Contraexemplos (CEGIS)

O paradigma CEGIS, formalizado por Solar-Lezama (Solar-Lezama 2008), estrutura a síntese de software em um ciclo cooperativo entre dois componentes: um sintetizador e um verificador. O sintetizador gera um programa candidato, e o verificador busca entradas em que o programa falha (contraexemplos). Caso um contraexemplo seja encontrado, ele é adicionado à especificação para que o sintetizador corrija o programa.

Com o advento dos LLMs, o CEGIS passou a ser empregado como um mecanismo interativo de correção de código (Poesia and Goodman 2024). Em vez de solucionadores formais (como SMT solvers), o próprio modelo atua como sintetizador probabilístico, recebendo mensagens de erro de execução ou divergências de saída para refinar sua solução.

### 2.3 Reflexão e Raciocínio em LLMs

Diversos métodos propõem que modelos de linguagem reflitam sobre seus próprios erros antes de gerar novas tentativas. Abordagens como *Self-Refine* (Madaan et al. 2023) e *Reflexion* (Shinn et al. 2023) demonstraram que solicitar ao modelo um resumo verbal da falha pode aumentar a taxa de sucesso em problemas de raciocínio e programação. A estratégia AntiCheat avaliada neste artigo segue essa linha, combinando a reflexão verbal sobre invariantes com restrições explícitas contra regras pontuais.

### 2.4 Escalabilidade Psicométrica e Escala de Guttman

A análise psicométrica vem sendo progressivamente adotada para compreender a organização das habilidades de modelos de IA (Guttman 1944). A Escala de Guttman avalia se os itens de um teste seguem uma ordem cumulativa unidimensional de dificuldade: se uma tarefa é dominada por um modelo de habilidade média, ela também deve ser dominada por modelos mais avançados. Aplicar essa análise ao ARC-AGI permite verificar se o benchmark reflete uma hierarquia de inteligência consistente ou se o desempenho dos modelos é fragmentado e desordenado.

## 3 Abordagem Proposta

Nesta seção, descrevemos como formulamos o problema de síntese de código para o ARC-AGI, o funcionamento do ciclo CEGIS básico e a inclusão das regras anti-ajuste da estratégia AntiCheat.

### 3.1 Formulação do Problema

Cada tarefa do ARC é composta por um conjunto de exemplos de treino  $D_{\text{train}} = \{(I_1, O_1), \dots, (I_K, O_K)\}$  e um ou mais pares de teste  $(I_{\text{test}}, O_{\text{test}})$ , onde cada  $I$  e  $O$  são matrizes bidimensionais com números inteiros no intervalo  $[0, 9]$ .

O objetivo é encontrar uma função em Python, denotada por  $f(\text{grid})$ , que satisfaça perfeitamente todos os exemplos de treino:

$$f(I_k) = O_k, \quad \forall k \in \{1, \dots, K\} \quad (1)$$

e que consiga generalizar para o exemplo de teste oculto:

$$f(I_{\text{test}}) = O_{\text{test}} \quad (2)$$

Para evitar o uso de bibliotecas externas complexas que poderiam mascarar o raciocínio do modelo, impõe-se a regra de utilizar exclusivamente tipos e funções nativas do Python (como listas, dicionários, conjuntos, laços e funções básicas como `len`, `range` e `min`).

### 3.2 O Ciclo CEGIS Padrão

O algoritmo CEGIS opera em até  $T_{\max} = 5$  iterações. O processo funciona em quatro etapas simples:

1. **Primeira Tentativa (Baseline):** O modelo de linguagem recebe todos os pares de treino formatados em JSON e produz a primeira versão da função `transform(grid)`.
2. **Execução e Avaliação:** O código gerado é executado de forma segura em cada exemplo  $(I_k, O_k)$ . Se o código produzir a saída correta para todos os  $K$  exemplos de treino, o ciclo é interrompido com sucesso no treino (*convergência*).
3. **Deteção do Contraexemplo:** Caso o código erre em algum exemplo ou sofra um erro de execução, o primeiro exemplo falho é selecionado como contraexemplo.
4. **Feedback e Reparo:** Uma nova mensagem é enviada ao modelo contendo a entrada  $I_k$ , a saída esperada  $O_k$ , a saída incorretamente produzida (ou a mensagem de erro) e a instrução para corrigir o código. O ciclo se repete até atingir a convergência ou esgotar as 5 iterações.

### 3.3 A Estratégia AntiCheat

A estratégia AntiCheat introduz duas mudanças fundamentais no prompt de feedback quando ocorre uma falha:

1. **Proibição de atalhos e regras pontuais:** A instrução proíbe explicitamente que o modelo use desvios condicionais (`if/else`) voltados a índices específicos de exemplos ou coordenadas fixas de uma grade isolada. O modelo é instruído a produzir uma regra matemática ou geométrica uniforme que se aplique igualmente a todas as grades.
2. **Enunciação prévia de invariantes:** Antes de gerar o bloco de código Python, o modelo é obrigado a explicar em texto qual é o *invariante abstrato* que falhou na versão anterior e como a nova lógica atende a todos os exemplos conhecidos sem criar casos especiais.

### 3.4 Algoritmo Resumido

O fluxo unificado de execução é apresentado no Algoritmo 1.

### 3.5 O Que Há de Novo

A principal novidade desta proposta é o acoplamento entre um verificador determinístico estrito de código (que não permite aproximações) e restrições semânticas que impedem o modelo de trapacear na correção de bugs. Em vez de permitir que o LLM resolva o erro por força bruta sintática, forçamos a geração de uma hipótese conceitual antes da escrita da lógica em Python.

## 4 Metodologia Experimental

### 5 Resultados

Nesta seção, apresentamos os resultados obtidos na avaliação de 8 modelos de linguagem em 100 tarefas do ARC-AGI, totalizando 800 pares de execução. Analisamos a acurácia das abordagens, o comportamento qualitativo do código, a estrutura de dificuldade das tarefas, a complexidade do código gerado e a dinâmica do ciclo iterativo.

#### Algoritmo 1: Ciclo CEGIS com opção de regras AntiCheat

```

1 Entrada: Pares de treino  $D_{\text{train}} = \{(I_k, O_k)\}_{k=1}^K$ , Modo
   AntiCheat  $\in \{\text{verdadeiro, falso}\}$ , Limite  $T_{\max} = 5$ 
2 Saída: Programa final  $f^*$  e status de convergência
3  $f_1 \leftarrow \text{GerarPrimeiraTentativa}(D_{\text{train}})$ ;
4 para  $t \leftarrow 1$  até  $T_{\max}$  faça
5   falhas  $\leftarrow \emptyset$ ;
6   para cada  $(I_k, O_k) \in D_{\text{train}}$  faça
7      $\hat{O}_k \leftarrow \text{Executar}(f_t, I_k)$ ;
8     se  $\hat{O}_k \neq O_k$  então
9       falhas  $\leftarrow \text{falhas} \cup \{(k, I_k, O_k, \hat{O}_k)\}$ ;
10      interromper;
      // Seleciona o primeiro
      contraexemplo
11   se falhas =  $\emptyset$  então
12     retorna  $f_t$ , convergiu = verdadeiro;
13    $(idx, I, O, \hat{O}) \leftarrow \text{Primeiro}(\text{falhas})$ ;
14   prompt  $\leftarrow \text{MontarFeedback}(idx, I, O, \hat{O})$ ;
15   se AntiCheat é verdadeiro então
16     prompt  $\leftarrow$ 
17       prompt + RegrasDeInvarianteEAntiAtalhos();
18    $f_{t+1} \leftarrow \text{ChamarModelo}(\text{prompt})$ ;
19 retorna  $f_{T_{\max}}$ , convergiu = falso;

```

### 5.1 Comparação de Acurácia: CEGIS vs. AntiCheat

A Tabela 1 resume o desempenho de cada modelo sob as duas estratégias. No total de 800 avaliações, o CEGIS acertou 403 tarefas (50,4%), enquanto o AntiCheat acertou 398 tarefas (49,8%), representando uma diferença absoluta de apenas 5 tarefas (−0,62 pontos percentuais).

Table 1: Desempenho por modelo no CEGIS e no AntiCheat (100 tarefas por modelo).

| Modelo                 | CEGIS (%)    | AntiCheat (%) | Dif. (%)     | Vitórias |
|------------------------|--------------|---------------|--------------|----------|
| Grok 4.6               | 92,0%        | 92,0%         | 0,0%         | 1        |
| DeepSeek V4 Flash 0731 | 86,0%        | 86,0%         | 0,0%         | 1        |
| GPT-5.6 Luna           | 77,0%        | 79,0%         | +2,0%        | 1        |
| Claude Sonnet 5        | 62,0%        | 60,0%         | -2,0%        | 1        |
| Gemini 3.5 Flash Lite  | 43,0%        | 45,0%         | +2,0%        | 1        |
| Gemini 3.1 Flash Lite  | 31,0%        | 24,0%         | -7,0%        | 1        |
| Leantral 1.5.1         | 7,0%         | 9,0%          | +2,0%        | 1        |
| GPT-5.4 Nano           | 5,0%         | 3,0%          | -2,0%        | 1        |
| <b>Total Agregado</b>  | <b>50,4%</b> | <b>49,8%</b>  | <b>-0,6%</b> | <b>1</b> |

Para verificar se essa diferença tem significado estatístico, aplicamos testes pareados nos pares discordantes (casos em que um método acertou e o outro errou):

- **Teste Binomial Exato:** Dos 45 casos de discordância, o CEGIS venceu 25 e o AntiCheat venceu 20. O teste binomial exato resulta em  $p = 0,5515$ .
- **Teste de Permutação:** Com 100.000 permutações de sinal pareadas, obtivemos  $p = 0,5501$ .

- **Intervalo de Confiança (*Bootstrap*):** O intervalo de confiança de 95% com 10.000 reamostragens ficou entre  $[-2, 25\%, +1, 00\%]$ .

Como o valor de  $p$  é substancialmente superior a 0,05 e o intervalo de confiança inclui o zero, **não há evidência estatística de que a estratégia AntiCheat seja superior ou inferior ao CEGIS**.

Em relação ao modelo Gemini 3.1 Flash Lite, que apresentou uma queda aparente de 7% (7 acertos exclusivos do CEGIS contra 0 do AntiCheat, com  $p = 0,0156$  não corrigido), a aplicação da correção de Bonferroni para os 8 modelos avaliados eleva o  $p$ -valor para 0,1250. Esse resultado neutraliza a diferença, indicando que a variação observada é compatível com a oscilação esperada do acaso ao testar múltiplas hipóteses simultaneamente, não se configurando como uma regressão inesperada.

## 5.2 Análise Qualitativa do Código e Motivos de Falha

Buscamos entender se, além dos números de acurácia, as regras do AntiCheat provocavam alterações qualitativas relevantes nos programas gerados. Inspecionamos detalhadamente os 45 casos discordantes e os motivos de falha de cada estratégia:

1. **Motivo de falha predominante:** Nos 25 casos em que o CEGIS venceu, a falha do AntiCheat decorreu de teto de representação (*representation ceiling*) em 23 deles (92,0%). Da mesma forma, nos 20 casos em que o AntiCheat venceu, o CEGIS falhou por teto de representação em 18 deles (90,0%). Em ambos os casos, a estratégia perdedora simplesmente não conseguiu encontrar uma lógica que satisfizesse os exemplos de treino no limite de 5 iterações.
2. **Ocorrência de sobreajuste espúrio:** O sobreajuste espúrio (acertar o treino, mas errar o teste) ocorreu em apenas 2 tarefas de cada lado.
3. **Ausência de trapaças por coordenadas:** Uma varredura estática em todas as 2.398 funções geradas no benchmark revelou que apenas 7 códigos (0,29%) continham coordenadas fixas ou condicionais direcionadas a exemplos isolados.

A inspeção do código-fonte revelou que os dois métodos produzem soluções estruturalmente muito próximas. O modelo mantém a mesma estratégia algorítmica básica em ambos os casos, e pequenas variações no prompt afetam apenas detalhes pontuais de indexação ou busca geométrica. Portanto, **não se observa diferença qualitativa sistemática** entre o código do CEGIS e o código do AntiCheat.

## 5.3 Dificuldade das Tarefas e Escala de Guttman

A distribuição do número de modelos que resolveram cada uma das 100 tarefas é apresentada na Figura 1. Observamos que:

- **Tarefas Universalmente Resolvidas:** Apenas 3 tarefas (3%) foram resolvidas por todos os 8 modelos (0c786b71, 2072aba6, 332efdb3).

- **Tarefas Universalmente Não Resolvidas:** 7 tarefas (7%) não foram solucionadas por nenhum dos modelos avaliados.
- **Armadilhas Indutivas:** Foram identificadas 5 tarefas em que a maioria dos modelos atingiu 100% de acerto nos treinos, mas falhou totalmente no teste (por exemplo, a tarefa 351d6448, onde 6 dos 8 modelos convergiram no treino, mas nenhum acertou o teste).

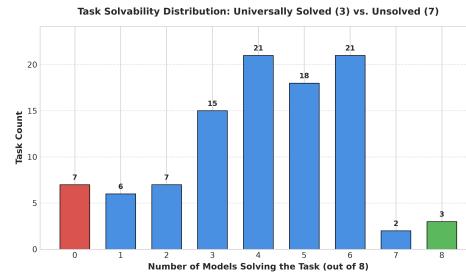


Figure 1: Distribuição de tarefas resolvidas por quantidade de modelos.

Para analisar se o benchmark mede uma habilidade cumulativa única, calculamos a Matriz de Subsunção de Guttman, ilustrada na Figura 2. A probabilidade de um modelo superior acertar as tarefas que um modelo inferior acerta é expressiva:

- Grok 4.6 resolveu **100,0%** das tarefas resolvidas pelo DeepSeek V4 Flash;
- DeepSeek V4 Flash resolveu **97,4%** das tarefas resolvidas pelo GPT-5.6 Luna;
- GPT-5.6 Luna resolveu **91,9%** das tarefas resolvidas pelo Claude Sonnet 5;
- Claude Sonnet 5 resolveu **86,0%** das tarefas resolvidas pelo Gemini 3.5 Flash Lite.

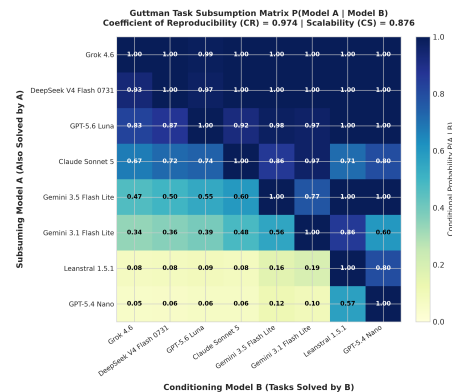


Figure 2: Matriz de Subsunção de Guttman entre os 8 modelos avaliados.

Os índices psicométricos calculados confirmam que o ARC-AGI forma uma escala unidimensional quase perfeita:

- **Coefficiente de Reprodutibilidade (CR):** 0,9738 (o limiar de aceitação psicométrico é  $\geq 0,90$ );

- **Coefficiente de Escalabilidade (CS):** 0,8757 (o limiar de aceitação é  $\geq 0,60$ ).

Isso confirma que o ganho de desempenho dos modelos segue uma progressão estritamente hierárquica de competências.

#### 5.4 Complexidade do Código e a Navalha de Occam

Analizamos a Árvore Sintática Abstrata (AST) dos programas gerados para investigar a relação entre o tamanho do código e o sucesso da solução (Figura 3).

Table 2: Complexidade estrutural do código por desfecho da execução.

| Categoria de Desfecho                               | Amostras | Linhas (SLOC) |
|-----------------------------------------------------|----------|---------------|
| Acerto de Primeira ( <i>both correct</i> )          | 289      | 34,6          |
| Recuperado via CEGIS ( <i>semantic recovery</i> )   | 114      | 44,5          |
| Sobreajuste Espúrio ( <i>spurious overfitting</i> ) | 52       | 55,9          |
| Falha no Treino ( <i>ceiling</i> )                  | 342      | 44,4          |

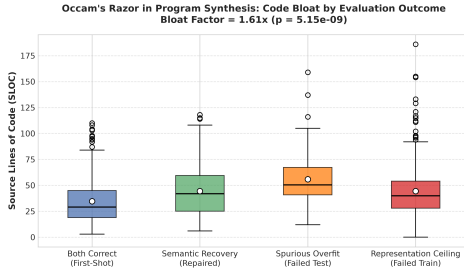


Figure 3: Distribuição das linhas de código (SLOC) de acordo com o desfecho da tarefa.

Como resumido na Tabela 2, os códigos que acertam na primeira tentativa possuem média de 34,6 linhas de código. Em contraste, os códigos que caem em sobreajuste espúrio apresentam uma média de 55,9 linhas. Esse aumento representa um fator de inchaço (*Bloat Factor*) de **1,61 vezes** ( $p = 5,15 \times 10^{-9}$  no teste de Mann-Whitney U). Esse resultado fornece uma validação empírica rigorosa do princípio da **Navalha de Occam**: quando os modelos tentam acomodar contraexemplos sem entender o padrão geral, tendem a inflar o programa com condicionais e laços adicionais que não generalizam.

#### 5.5 Dinâmica de Iterações do CEGIS

Avaliamos quantas iterações são necessárias para que o CEGIS conserte códigos que falharam na primeira tentativa. Conforme apresentado na Tabela 3 e na Figura 4:

- 71,7% (289 tarefas) do total de acertos ocorrem na primeira iteração;
- Para as 114 tarefas corrigidas ao longo do ciclo iterativo, 60,5% das correções acontecem logo na 2ª iteração e 25,4% ocorrem na 3ª iteração;
- As iterações 4 e 5 respondem por apenas 7,9% e 6,1% das correções, respectivamente.

Table 3: Distribuição de recuperação de tarefas ao longo das iterações do CEGIS.

| Iteração | Tarefas Resolvidas | % do Total Resolvido | Tarefas Recuperadas |
|----------|--------------------|----------------------|---------------------|
| 1        | 289                | 71,7%                | 0                   |
| 2        | 69                 | 17,1%                | 69                  |
| 3        | 29                 | 7,2%                 | 29                  |
| 4        | 9                  | 2,2%                 | 9                   |
| 5        | 7                  | 1,7%                 | 7                   |

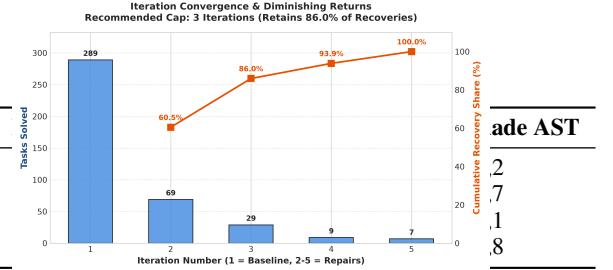


Figure 4: Rendimentos decrescentes ao longo das iterações do CEGIS.

Esse comportamento demonstra um retorno decrescente rápido. Fixar o limite do CEGIS em **3 iterações** permite capturar 86,0% de todos os reparos possíveis, evitando o consumo desnecessário de tokens e processamento nas rodadas finais.

## 6 Discussão e Conclusões

### 6.1 Síntese dos Resultados e Interpretação

Neste trabalho, investigamos se a introdução de restrições semânticas explícitas contra regras pontuais e a obrigatoriedade de explicar invariantes (*AntiCheat*) trariam ganhos reais sobre a síntese iterativa de programas por CEGIS no benchmark ARC-AGI.

A principal conclusão empírica é que a estratégia *AntiCheat* **não produziu diferença estatisticamente significativa** na acurácia geral (50,4% contra 49,8%, com  $p = 0,5515$ ). Além disso, a inspeção qualitativa confirmou que essa equivalência ocorre porque as falhas em ambos os métodos são causadas quase que exclusivamente pelo mesmo motivo: o esgotamento da capacidade do modelo de formular o algoritmo correto (*representation ceiling*), e não pela tentativa deliberada de criar atalhos com coordenadas fixas. Em modelos avançados de linguagem, a criação de atalhos pontuais de coordenadas é um fenômeno residual (presente em menos de 0,3% dos códigos analisados).

Por outro lado, nossa análise estrutural produziu descobertas relevantes e independentes de estratégias específicas:

1. **Consistência Psicométrica:** O ARC-AGI exibe uma organização hierárquica de dificuldade quase perfeita na Escala de Guttman ( $CR = 0,9738$ ), mostrando que o benchmark reflete uma dimensão estável de avanço de capacidades dos modelos.

2. **Validação da Navalha de Occam:** Programas que caem em sobreajuste sofrem uma inflação sistemática no número de linhas (1,61 vezes maiores,  $p = 5,15 \times 10^{-9}$ ), laços e desvios condicionais, confirmando numericamente que soluções simples têm maior probabilidade de generalização.
3. **Horizonte de Eficiência:** Estabelecemos que limitar o ciclo de correção do CEGIS a 3 iterações preserva 86,0% dos reparos viáveis, otimizando o custo computacional.

## 6.2 Aplicações e Impactos

Estes achados trazem orientações diretas para o desenvolvimento de sistemas de programação assistida por IA:

- **Crítérios de seleção por simplicidade:** Sistemas de busca e geração de código devem usar o tamanho da AST e o número de linhas como função de penalidade para descartar programas que se tornam excessivamente complexos após múltiplas tentativas de correção.
- **Crítérios de parada precoce:** Encerrar o ciclo CEGIS após 3 iterações é uma política eficiente para evitar gasto de recursos em soluções que raramente convergem.

## 6.3 Limitações

As principais limitações deste estudo incluem:

1. **Amostragem de tarefas:** A avaliação utilizou uma amostra representativa de 100 tarefas do conjunto de treino/avaliação do ARC-AGI.
2. **Domínio restrito ao Python nativo:** A regra de não permitir bibliotecas externas padroniza o teste, mas pode restringir o estilo de solução dos modelos.
3. **Temperatura e amostragem única:** Cada tarefa foi executada com parâmetros fixos de amostragem, sem exploração massiva de sementes aleatórias ( $pass@k$ ).

## 6.4 Trabalhos Futuros

Como perspectivas de trabalhos futuros, sugerimos:

- **Variação sistemática entre prompts:** Investigar como diferentes estilos de instrução, exemplos no prompt (*few-shot prompting*) e formulações alternativas de regras afetam a capacidade do modelo de se autorregular.
- **Políticas adaptativas de correção:** Desenvolver mecanismos que decidem o tipo de feedback com base na complexidade do erro observado na execução anterior.
- **Filtros de seleção sintática:** Aplicar limites rígidos de linhas de código durante a síntese para barrar automaticamente códigos que sofram o inchaço característico do sobreajuste.

## References

- Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.
- Greenblatt, R.; Shlegeris, B.; Korbak, T.; and Barnes, E. 2024. Getting 50% (and more) on ARC-AGI with Large Language Models and Program Search. *arXiv preprint arXiv:2412.04604*.
- Guttman, L. 1944. A basis for scaling qualitative data. *American Sociological Review*, 9(2): 139–150.
- Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhume, S.; Yang, Y.; et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36: 46534–46594.
- Poesia, G.; and Goodman, N. D. 2024. Toward Next-Generation Program Synthesis: Neurosymbolic Approaches and Large Language Models. *Communications of the ACM*.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36: 8634–8652.
- Solar-Lezama, A. 2008. *Program Synthesis by Sketching*. Ph.D. thesis, University of California, Berkeley.